

# ControlNet の制御が効かなくなる条件と、その機構

映像メディア学 レポート課題

新井 滉明 (Komei Arai) / 学籍番号 48266454

東京大学大学院 情報理工学系研究科 電子情報学専攻 修士1年 / 伊庭研究室

研究テーマ: 進化計算・学習における資源の会計

ソースコード: <https://github.com/komeiall14/controlnet-failure-analysis>

**本レポートで示したこと。** ControlNet の制御が壊れる条件を 3 つ切り出し、なぜ壊れるかまで実験で示した。推奨されている省メモリ設定が黙って出力を壊す件は、`get_attention_scores` が未初期化のまま渡す加算元が原因で、0 で埋めるだけで直る。条件地図の密度が下がると制御が消える件では、指定した輪郭との一致が 0.7329 から 0.2391 へ落ちる一方、生成画像自体は破綻しない。UNet へ注入される残差を直接測ると、制御が消えるのは残差が消えるからではなく条件地図に由来する成分が縮むからだと分かる。ここから密度を揃える前処理を作り、開発に使っていない画像でも効果を確かめた。三つ目の既定値が弱い件は、破綻ではなく二つの指標のあいだの妥協点として第 8 節で扱う。

## 1. 対象論文

Lymin Zhang, Anyi Rao, Maneesh Agrawala.

"Adding Conditional Control to Text-to-Image Diffusion Models."

Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 3813–3824. DOI: 10.1109/ICCV51070.2023.00355 (本会議の full paper. 公開モデル [lllyasviel/sd-controlnet-canny](https://github.com/lllyasviel/sd-controlnet-canny) が利用できる)

この論文が扱うのは、テキストだけでは指定できない条件を、学習済みの巨大な拡散モデルに後付けする問題である。「猫の絵」はプロンプトで頼めるが、「この輪郭に沿った猫」は言葉にならない。素朴には条件つきで再学習すればよいが、論文が障害として挙げるのはデータ量で、特定の条件では最大級のデータセットでも約 10 万枚と LAION-5B の 5 万分の 1 しかなく、その量で直接微調整すれば過学習と破滅的忘却を招くと述べている。

ControlNet の答えは、元のモデルには一切手を触れないというものである。重みを凍結したまま、そのエンコーダと中間ブロックの複製を作って条件を受け取らせ、複製の出力を zero convolution という 0 で初期化した層を通して元のモデルへ残差として加える。同じ枠組みで条件の種類を差し替えられるのも特徴で、輪郭・深度・姿勢・領域分割がそれぞれ別の分岐として公開されている。本レポートが扱うのは輪郭 (Canny エッジ) による条件づけである。

## 2. 手法理解

ControlNet は学習済み拡散モデルの重みを凍結したまま、そのエンコーダ 12 ブロックと中間ブロックの複製 (trainable copy) を作り、両者を zero convolution で接続する。複製の出力は元の U-Net の 12 本のスキップ接続と中間ブロックに加算される。論文が与える式は

$$y_c = F(x; \Theta) + Z(F(x + Z(c; \Theta_{z1}); \Theta_c); \Theta_{z2})$$

である。ここで  $F$  は凍結された元のブロック、 $\Theta$  はその重み、 $c$  が条件 (本レポートでは Canny エッジ地図)、 $Z$  が zero convolution で、 $\Theta_c \cdot \Theta_{z1} \cdot \Theta_{z2}$  が複製側と zero convolution の重みである。 $Z$  は重みとバイアスを 0 に初期化した  $1 \times 1$  畳み込みなので、学習開始時点では両方の  $Z$  が 0 を返し、 $y_c = F(x; \Theta)$  すなわち元のモデルの出力と完全に一致する。有害な雑音が微調整に影響しないよう、パラメータを 0 から徐々に育てるためだと論文は述べている。

残差の重み付けは適用条件を押さえておく必要がある。論文が解像度に応じた重み  $w_i = 64/h_i$  を導入するのは classifier-free guidance に関する箇所、条件画像を条件側の予測だけに加える場合の手当てである。常に全接続へ掛けるとは書かれていない。diffusers 0.31.0 もこれに対応し、`guess_mode` のときだけこの重みを掛け、既定では全ブロックに `controlnet_conditioning_scale` を一様に掛ける (`models/controlnet.py` L844-851)。移植の欠落ではない。本レポートは `guess_mode` を既定の無効のまま実験しているので、第 5 節で段ごとの残差を比べるとときに重みの補正は要らない。推論時に露出しているつまみは `controlnet_conditioning_scale` (既定 1.0) で、上式の第 2 項に掛かる係数にあたる。

学習の設定で本レポートに直接関わるのは、プロンプトの 50% を空文字に置き換えて学習している点である。論文はその狙いを「条件画像の意味を直接認識する能力を高めるため」と述べている。つまりこのモデルは、テキストが無いときには条件地図そのものから意味を読み取るよう仕向けられている。

両方の  $Z$  が 0 を返すのは学習開始の時点だけである。学習後の zero convolution は重みが育っているの  
で、条件が空でも 0 を返すとは限らない。加えて第 2 項は  $Z(F(x + Z(c; \Theta_{z1}); \Theta_c); \Theta_{z2})$  であり、条件が消  
えても潜在変数  $x$  が trainable copy を通る成分は残る。したがって条件が情報を失ったときに起きるのは、  
第 2 項が 0 になることではなく、条件地図に由来する成分が失われて残差の大きさが縮むことだと予測で  
きる。学習が「条件地図から意味を読む」方向に偏っている以上、失われるのは輪郭の指定だけではないと  
も予測できる。第 5 節でこの退化を残差の大きさとして測る。

論文自身のアブレーションが、この構造の二つの側面を示す。zero convolution を通常の畳み込みに置き  
換えると性能が軽量版と同程度まで落ち、論文はその理由を、微調整の途中で trainable copy の事前学習済  
みバックボーンが壊れるためだと説明する。一方その軽量版は、条件画像を解釈するには力不足でプロンプ  
トが不十分な条件では失敗するとも述べている。制御の強さは追加ネットワークの表現力と、事前学習済み  
の表現を壊さずに残差を渡せるかの両方で決まり、zero convolution が担うのは後者である。

なお、この論文には Limitations に相当する節が無い。arXiv 版 v3 の全文を検索しても limitation の語は  
一度も現れず、Discussion が挙げる三点はいずれも能力の側の話である。本レポートが第 7 節に挙げる限  
界は、すべて本実験から見出したものである。

3. 実行環境

|        |                                                                                                          |
|--------|----------------------------------------------------------------------------------------------------------|
| 計算機    | Apple M1 (8コアGPU) / RAM 16GB / macOS 14.4.1                                                              |
| バックエンド | PyTorch 2.2.2 の MPS (CUDA なし)                                                                            |
| ライブラリ  | diffusers 0.31.0 / transformers 4.44.2 / OpenCV 4.11.0 /<br>SciPy 1.16.0 / scikit-image 0.22.0 (入力画像の出所) |
| モデル    | Stable Diffusion v1.5 + sd-controlnet-canny (fp16、計<br>$1.43 \times 10^9$ パラメータ)                         |
| サンブラ   | DPMSolver++ 2M (Karras) 20ステップ、 $512 \times 512$                                                         |
| 生成速度   | 1枚あたり 76~184秒 ( $n = 157$ 、中央値 106.5秒)。第4.1<br>節の切り分けは DDIM 4ステップなので別条件である                               |

4. Failure Case Analysis

三つの性質は違う。4.1 は実行系の欠陥で、推奨されている設定に従うと沈黙して壊れる。4.2 と 4.3 は  
条件づけの機構そのものの性質である。

4.1 推奨されている省メモリ設定が、沈黙したまま出力を壊す

最初の生成が一枚も成立しなかった。返ってきた画像は  $512 \times 512$  の全画素が値 0、完全な黒である。例  
外も警告も出ず処理は正常終了する。VAE で復号する前の潜在変数を取り出すと、その段階ですでに NaN  
が入っていた。fp32 の VAE で復号し直しても消えないので、fp16 の VAE が飽和したという既知の話では  
なく、UNet の順伝播が壊れている。

原因は enable\_attention\_slicing() だった。発生条件を絞るため、精度 2 通り  $\times$  slicing の設定 5 通りで潜  
在変数の NaN を直接調べた (DDIM 4 ステップ、同一シード)。

|      | slicing なし | auto      | max       | 1         | 4         |
|------|------------|-----------|-----------|-----------|-----------|
| fp16 | 正常 (3.38)  | NaN       | NaN       | NaN       | NaN       |
| fp32 | 正常 (4.46)  | 正常 (4.46) | 正常 (4.46) | 正常 (4.46) | 正常 (4.46) |

括弧内は潜在変数の絶対値の最大である。fp16 と slicing の組み合わせでのみ発生し、分割の粒度に依存  
しない。fp32 では slicing を有効にしても差は小数第 6 位にとどまる。なお auto はヘッド数 (このモデル  
では 8) の半分、max は 1 に解決されるので、実際に試した粒度は 1 と 4 の 2 通りである。

diffusers 0.31.0 では slicing の有無で attention の実装が入れ替わる。既定は  
F.scaled\_dot\_product\_attention、slicing 有効時はスコア行列を構成する Attention.get\_attention\_scores を  
通り、後者は加算元を未初期化のまま確保する。

buf = torch.empty(...); scores = torch.baddbmm(buf, query, key.T, beta=0, alpha=scale)

beta = 0 なので加算元は参照されないはずで、BLAS の規約でもそう定められている (文献[6])。未初  
期化のまま渡す実装自体は規約に従っている。溢れの線は薄い。入力 of 絶対値の最大は 8.6 で、最も次元の  
大きい段で蓄積して係数を掛けても上限は 840 程度、fp16 の上限 65504 には遠く及ばない。

加算元を差し替えると挙動が変わる。torch.empty を torch.zeros にするだけで NaN が消え、潜在変数の絶対値の最大は 3.379~3.383 と slicing 無効時の 3.377 に近い値へ戻る。中身を直接測ると、fp16 では 2944 回のうち 2847 回のバッファが非有限値を含み、fp32 は 0 回だった (exp10\_buffer\_content.py)。規約が守られているかも調べた (exp11\_beta\_zero.py)。同じ入力で加算元を 0 と NaN に入れ替えると、MPS では結果が一致せず CPU では一致する。fp16 に限らず fp32 でも一致しない。加算元が有限値なら MPS でも一致するので、壊れるのは非有限値のときだけである。MPS の baddbmm は  $\beta = 0$  でも  $0 \times C$  を実際に計算しており、IEEE 754 の  $0 \times \text{NaN} = \text{NaN}$  がそのまま出る。

規約が保証するはずの「 $\beta = 0$  なら加算元は参照されない」が MPS で成り立たない。これが欠陥の中身である。fp16 でだけ壊れて見えるのは演算の違いではなく、その時点のメモリに非有限値が残っていたかどうかの違いにすぎない。

利用者の側で直すなら、加算元を 0 で初期化した実装に差し替えれば足りる。本番と同じ 20 ステップで確かめると、slicing を有効にしたままでも 5 枚すべてが正常に生成され、構造整合 (定義は第 4.2 節) の平均は 0.7426 で、slicing 無効時の 0.7427 との差は 0.0001 にとどまる。画素差も平均 0.18 (0~255 階調) で、実質的に同じ画像が出る。ただし slicing 本来の目的である省メモリ効果が得られているかは測っていない。取得できるのはプロセス全体の確保量だけで、3 条件とも同じ値になり判定できない。fp32 に切り替えても回避できるが、4 ステップで比べると fp16 の 23.6 秒に対し 39.9 秒と 1.7 倍遅い。以降の実験は fp16 のまま slicing を無効にして行った。

問題は、この設定が推奨されている点にある。diffusers の MPS 最適化に関する公式文書は RAM 64GB 未満で enable\_attention\_slicing() の有効化を案内しており、本環境の 16GB は条件に当てはまる。案内どおりに従うと生成が全滅し、しかも失敗が静かなので利用者は自分の設定ミスを疑う。静かに壊れる失敗は下流のすべての指標を無意味にするので、生成直後に NaN と単色を検出して例外を投げる検査も組み込んだ。

## 4.2 条件地図の密度が下がると制御が消える

Canny の閾値は公式実装の既定値 (100, 200) に固定したまま、入力のコントラストだけを係数  $\alpha$  で落とした系列を作った。構図と内容は変えていないので、変化するのは条件地図であり、その量をエッジ画素率で要約する。実画像 10 点 (skimage.data 同梱) と、円と矩形の反復数だけを変えた合成画像 5 点に対し、 $\alpha \in \{0.15, 0.3, 0.5, 1.0\}$  を掛けた 60 条件を生成した。

構造整合は、生成画像から条件と同じ設定で Canny を再抽出し、入力の条件地図との F1 で測る (2 画素の許容を入れた膨張マッチング。拡散生成では輪郭が数画素揺れるので、許容なしでは構造が保たれていても F1 がほぼ 0 になる)。

| 条件地図のエッジ画素率 | n  | edge F1 平均 | 中央値   | 標準偏差  |
|-------------|----|------------|-------|-------|
| 0 (完全に消滅)   | 15 | —          | —     | —     |
| ~0.005      | 6  | 0.109      | 0.012 | 0.229 |
| ~0.02       | 11 | 0.668      | 0.611 | 0.288 |
| ~0.05       | 14 | 0.721      | 0.748 | 0.145 |
| 0.05~       | 14 | 0.841      | 0.859 | 0.088 |

密度 0 の行を「—」としたのは、この指標がそこでは使えないからである。条件地図が空だと適合率は空虚に 0、再現率は 0/0 になり、生成画像が何であっても F1 が 0 を返す。「密度 0 で F1 が 0」は破綻の測定ではなく、指標の定義から自動的に出る値である。破綻の証拠にはできない。密度が正の 45 条件に限れば、エッジ画素率と構造整合の順位相関は  $\rho = +0.577$  ( $p = 3.4 \times 10^{-5}$ ) である。

代わりに 3 点を述べる。第一に、60 条件のうち 15 条件でエッジ画素率がちょうど 0 になった。既定の閾値が入力のコントラストに対して高すぎるためで、Canny の閾値処理だけで決まるのでシードには依存しない。ただし条件地図が空になると元画像が違ってもパイプラインへ渡る入力が一致するため、15 行のうち生成画像が異なるのは 8 枚だけで、独立な観測は 8 件と数えるべきである。

第二に、条件が効いていないことを示すには別の測り方が要る。密度 0 の生成画像を、本来指定したかった輪郭 (コントラストを落とさない条件地図) と照合すると F1 は平均 0.2391 で、正しく条件づけた生成の 0.7329 から落ちている。この落差が主張の中心である。無関係な画像どうしを組み合わせた偶然一致の水準は 0.2461 で、密度 0 の 0.2391 がこれを上回るとは言えない (Mann-Whitney U 検定、 $p = 0.807$ )。非有意は一致の証明ではないので、偶然の水準と同じだとまでは主張しない。重複を除いた 8 枚でも平均

0.2822 で結論は変わらない。

壊れているのは制御であって、生成ではない。密度 0 の生成画像はエッジ画素率 0.0074~0.2539 を持つ普通の画像で、真っ黒でも崩れてもいない。chelsea は猫、rocket はロケットとプロンプトの語に対応する物は出るが、図1の camera では主題の人物まで落ちる。失われるのは輪郭の指定だけとは限らない。

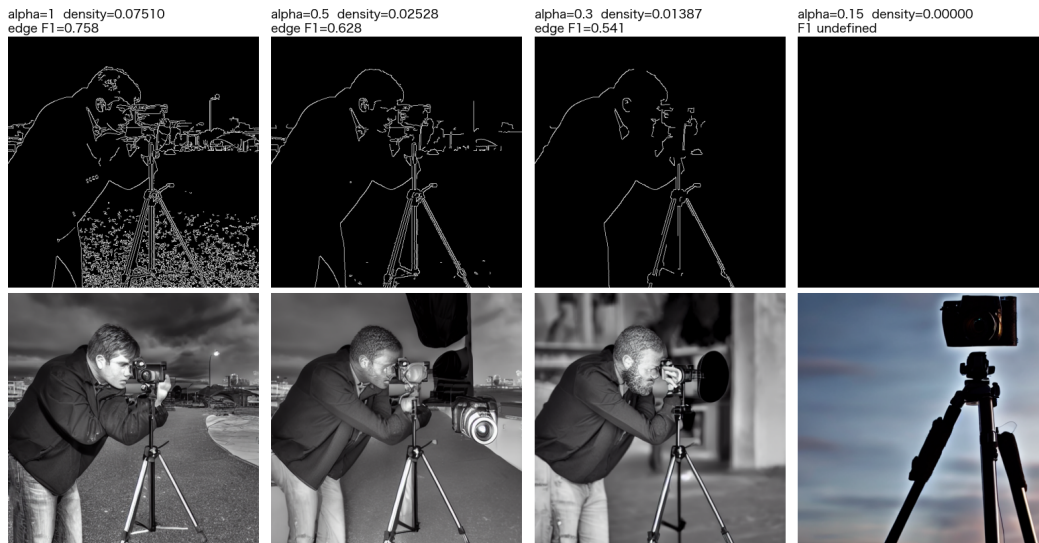


図1 同一画像のコントラストだけを落としたときの条件地図（上段）と生成結果（下段）。プロンプトは全列で同一。 $\alpha=0.15$  では条件地図が完全な黒（最大値 0）になるが、生成画像は破綻せず、人物が消えて構図が元画像と無関係になる。密度/F1 は左から 0.0751/0.758、0.0253/0.628、0.0139/0.541、0/定義不能。

第三に、低密度域は成績が悪いだけでなくばらつきが大きい。密度帯を代表する 5 条件を四分位で選び、シードだけを変えて 5 回ずつ生成した。条件内の標準偏差は平均 0.0644 で、条件平均の標準偏差 0.3543 の 5.5 分の 1 にとどまるので、密度の効果はシードの揺らぎでは説明できない。一方その条件内の標準偏差は密度で変わり、最も揺れる密度 0.0201 では 0.1433（密度 0.1884 では 0.0213）で、シードを変えるだけで edge F1 が 0.4965 から 0.8589 まで動く。壊れかけの領域では、制御が効くかどうか賭けになる。

### 4.3 推論時の既定値は構造整合に対して弱すぎる

当初は「最適な controlnet\_conditioning\_scale は入力の密度によって動く」という予想を立て、7 つの入力に対し  $scale \in \{0.2, 0.4, 0.6, 0.8, 1.0, 1.3\}$  を振った。この予想は外れた。7 入力すべてで掃引範囲の上限である 1.3 が最良となり、最適値が全件同一なので密度との順位相関そのものが定義できない。

外れ方から分かったこともある。既定値 1.0 が最適だった入力 7 件中 0 件で、どの入力でも 1.0 から 1.3 へ上げると構造整合が改善した。改善幅は入力によって大きく異なり、密度の低い synth\_sparse では 0.333 から 0.877 へ跳ねる一方、密度の高い synth\_veryd では 0.957 から 0.978 にとどまる。ただしこれは構造整合だけを見た結果で、掃引範囲を狭く取ったために代償が見えていない。第 8 節では範囲を 3.0 まで延ばし、プロンプト追従（CLIP 類似度）と併せて評価する。

## 5. 機構の直接証拠：注入される残差そのものを測る

第 4.2 節で示したのは入力と出力の対応にすぎない。密度が下がると成績が落ちるという相関は、条件が効かなくなったからだとも、生成そのものが難しくなったからだとも読める。第 2 節で立てた予測は前者、すなわち zero convolution を通って UNet に加わる残差が縮むというものだった。これは測れる。

ControlNet の順伝播を 1 回だけ実行し、UNet の各段へ渡される残差の二乗平均平方根を求めた。timestep は 500、潜在変数はシード 0 で固定した共通のものを使い、条件側の枝だけを見ている。ただし系列間ではプロンプトも変わるので、条件地図の違いだけを取り出した比較が厳密に成り立つのは各画像のコントラスト系列の内部である。拡散を回さないため 1 条件 1 秒未満で済み、コントラスト係数を 9 段階に振れた。

| 条件地図のエッジ画素率 | 順伝播の回数                | 総残差 RMS | mid ブロックの RMS |
|-------------|-----------------------|---------|---------------|
| 0（完全に消滅）    | 45（異なる値は 11。違いはプロンプト） | 0.3846  | 0.9333        |
| ~0.005      | 11                    | 0.5161  | 1.2990        |
| ~0.02       | 28                    | 0.8027  | 1.8046        |

| 条件地図のエッジ画素率 | 順伝播の回数 | 総残差 RMS | mid ブロックの RMS |
|-------------|--------|---------|---------------|
| ～0.05       | 27     | 0.9785  | 2.0590        |
| 0.05～       | 24     | 1.1114  | 2.0724        |

回数の列は独立な観測数ではない。順伝播は決定論的なので、同一の条件地図を与えた行はビット単位で一致する。また密度 0 の条件地図は全画素 0 の配列で、参照に用意した空白の地図と同一である。両者の残差が近いのは当然なので、この比較は主張に使わない。

機構として意味を持つのは、密度が正の側である。そこでの減衰を見ると総残差は 1.1114 から 0.5161 へ単調に下がる。全 135 回での順位相関  $\rho = +0.875$  も密度 0 の 45 回に押し上げられているので、密度が正の 90 回に限った  $+0.669$  を採る。さらに画像ごとにコントラスト系列の内部で相関を取ると、密度が実際に段階的に動く実画像 8 枚では、密度が正の範囲に限っても  $+0.943 \sim +1.000$  である（残る 7 枚は密度が 0 から一段跳ぶだけか、ほとんど動かないので系列内の勾配が取れない）。画像の内容を固定して密度だけを動かしても残差が縮むので、画像間の差では説明できない。

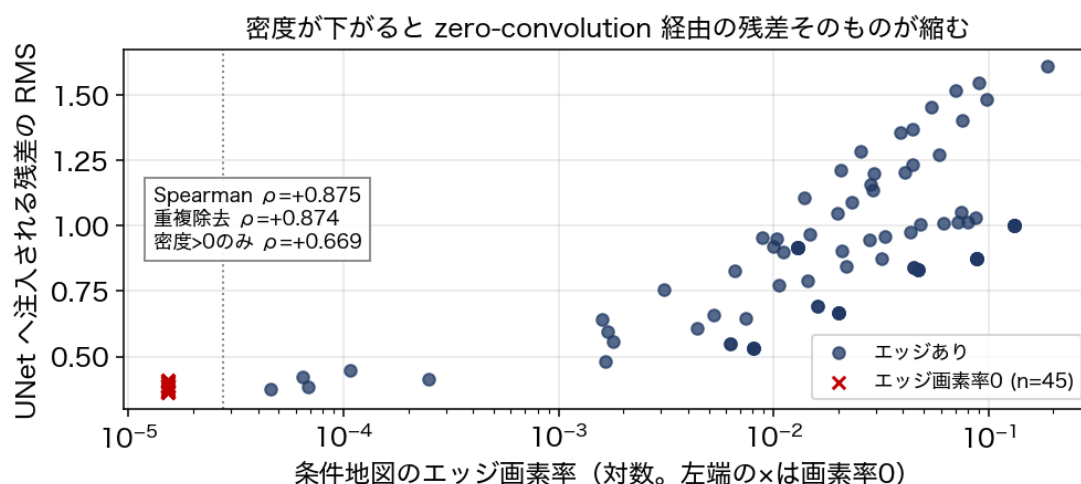


図2 条件地図のエッジ画素率と、UNet へ注入される残差の大きさ。点線より左は、対数軸に載らない画素率 0 の 45 回を別枠に置いたもので、横位置そのものに意味はない（全画素 0 の同一の地図なので値は 11 通り）。本文が機構の根拠に据えているのは、密度が正の領域の減衰である。

残ったものが何かも確かめた。残差を空間方向に分解し、チャンネルごとの空間平均を引いた成分の割合を測る (probe\_spatial.py)。条件が消えた camera ( $\alpha = 0.15$ ) では mid の RMS 0.9889 のうち 67.5% が空間成分で、残差は一樣ではない。原画像では mid の RMS が  $3.2323 \cdot$  空間成分 84.5% なので、条件が消えると残差は 3.3 分の 1 に縮み、空間的に変化している割合も下がる。条件が失われたときに起きるのは、残差が消えることでも一樣な項になることでもなく、条件地図に由来する成分が縮むことである。本節で測ったのは残差だけで、出力側で何が起きるかは第 4.2 節で別に測った。

## 6. 改善と、その前後比較

改善は二つある。一つは第 4.1 節の未初期化バッファの一行修正で、推論の前段が静かに壊れる問題を直す。もう一つが本節で扱う制御そのものの品質の改善である。第 4.2 節と第 5 節から、条件が効かなくなる原因は閾値が固定されていることに帰着する。入力のコントラストが変われば同じ閾値でもエッジ画素率が桁で動き、極端な場合は 0 になる。ならば閾値ではなく密度を揃えればよい。

実装は前処理と規則の二段である。前処理は、目標エッジ画素率 (0.045) へ近づくよう Canny の高閾値を 12 回二分探索し、その中で目標に最も近い密度の地図を採る。探索の前に CLAHE で局所コントラストを補正しておく。規則の側は、得られた密度に応じて controlnet\_conditioning\_scale を調整する。どちらも拡散を回す前の処理なので、拡散のステップ数は変わらない。

評価は同一の画像・プロンプト・シードで改善前と改善後を対にして行った。15 画像  $\times$  コントラスト 2 水準  $\times$  シード 3 個で 90 対である。検定の単位に注意が要る。1 枚から 6 行出るので 90 行をそのまま検定にかけると標本数が水増しされるため、シードは画像内で平均して画像を単位とした。コントラストの水準も混ぜてはいけない。混ぜると符号が打ち消し合う。参照地図の取り方にも落とし穴がある。前後をそれぞれが与えられた地図で測ると、F1 が参照地図の密度に依存するぶんが改善幅に混入するので、両者を同じ参照で測り直した値も併記する。



| コントラスト                   | n (画像) | 改善した画像  | $\Delta$ 中央値 | Wilcoxon p |
|--------------------------|--------|---------|--------------|------------|
| $\alpha = 0.3$ (低コントラスト) | 15     | 12 / 15 | +0.1194      | 0.041      |
| $\alpha = 1.0$ (原画像)     | 15     | 5 / 15  | -0.0784      | 0.188      |

低コントラスト側では改善し、原画像側では逆向きに振れている。二層を平均すると相殺されて「差がない」になるが、それは実態を隠す。共通参照で測り直すと  $\alpha = 0.3$  は  $\Delta$  中央値 +0.0665・p = 0.055、 $\alpha = 1.0$  は -0.0560・p = 0.107 で、改善の向きは保たれるが大きさは小さくなる。低コントラスト側で効くのは筋が通っている。改善前の平均密度 0.0270 が 0.0354 へ上がって目標に近づき、条件地図が空だった 2 画像では密度が 0 から平均 0.0346 へ回復した。

原画像側が悪化の向きに振れるのは、二分探索が目標に届いていないためだと考えられる。 $\alpha = 1.0$  の 15 画像のうち適応後の密度が目標の  $\pm 10\%$  に収まったのは 5 画像で、平均は 0.0647 から 0.0630 へしか動かない。高閾値の探索範囲を 20~400 に切っているため実画像では上端でも密度が落ちきらず、合成画像は密度が階段状に変わるので動かない。scale を決める規則は適応後の密度を読むので、高いまま残った密度から scale が平均 0.854 まで下がり、構造整合が下がる。

ここまで構造整合だけを見てきたが、それでは第 4.3 節と同じ誤りを繰り返す。生成済み画像に CLIP を後付けで測ると、プロンプト追従は  $\alpha = 0.3$  で 0.2911  $\rightarrow$  0.2967、 $\alpha = 1.0$  で 0.2931  $\rightarrow$  0.2984 と、どちらも下がってはいない。scale を大きく振れば追従は下がるが（第 8 節）、この改善が動かす幅では代償が出ていない。

推奨する形そのものも評価した。改善前の密度が目標を下回るときにだけ適用する一行の判定を足すと、 $\alpha = 0.3$  では 13 / 15 画像が対象になり共通参照での p が 0.055 から 0.0019 へ下がる。 $\alpha = 1.0$  では 6 / 15 にしか適用されず、悪化していた  $\Delta$  中央値が 0.0000 に戻る。害を止めたうえで、効く場面を残せている。ここまでは規則を導いたのと同じ 15 画像での評価なので、改善も適用条件も作るのに使っていない skimage.data の 11 画像を別に用意し、規則を変えずに当てた。低コントラスト側は 11 画像すべてが適用条件を満たすので無条件適用と同じ値になり、10 画像で改善して  $\Delta$  中央値 +0.0788・p = 0.010 だった。原画像側は無条件に適用すると 8 画像で悪化し、 $\Delta$  中央値 -0.0688・p = 0.024 と有意に害が出る。適用条件を入れると対象が 6 画像に絞られ、 $\Delta$  中央値は 0.0000 (p = 0.688) へ戻る。改善も、適用条件が害を止めることも、開発に使っていない入力で再現した。

## 7. Limitations

**指標の設計に依存している。** 構造整合は再抽出した Canny との F1 で測るが、2 画素の許容幅・再抽出時の閾値・膨張マッチングに依存する。条件地図が空だと定義から 0 を返すので、破綻そのものをこの指標では測れない。代替の偶然一致水準との比較も「本来指定したかった輪郭」を人為的に選んでいる。

**標本数が小さい。** 第 6 節の改善前後比較は片側 15 画像、シードは 3 個で、有意性が個々の画像の扱いで動く程度の検出力しかない。実際、指標が定義から壊れている画像を除くと  $\alpha = 0.3$  の p は 0.0040 まで下がりも 0.110 まで上がりもする。開発に使っていない 11 画像でも確かめたが、そちらも同程度の規模である。

**扱ったのは輪郭条件だけである。** 深度条件については、コントラストを完全に潰した極限で残差 RMS が 1.50 から 0.48 へ同じ向きに落ちるところまでしか測っていない。途中の水準ではほとんど動かないので、輪郭で見たような段階的な減衰までは確かめていない。姿勢・領域分割は扱っておらず、密度に相当する量を条件の種類ごとにどう定義するかという問題も残る。

**環境とモデルに固有の可能性がある。** 第 4.1 節の欠陥は PyTorch 2.2.2 の MPS で確かめた。CPU では規約が守られることも測ったが、CUDA など他のバックエンドや他の版は扱っていない。生成側も Stable Diffusion v1.5 系に限られる。ただし推奨設定に従うと例外を出さずに壊れる構図そのものは、環境が違って起こりうる。

**論文自身の主張を検証したものではない。** 扱ったのは推論時の使い方であり、論文が報告する学習の性質や他手法との比較は追試していない。ここで挙げた限界は、論文が述べたものではなく本実験から見出したものである。

## 8. 考察

第 4.3 節では scale を 1.3 まで振って、上げるほど構造整合が良くなるという結果を得た。そこで掃引を 3.0 まで延ばし、同時に CLIP 類似度でプロンプト追従を測った。

| scale (10 段階のうち 7 段階を抜粋) | 0.2   | 0.6   | 1.0   | 1.6   | 2.0   | 2.5   | 3.0   |
|--------------------------|-------|-------|-------|-------|-------|-------|-------|
| edge F1                  | 0.251 | 0.589 | 0.717 | 0.909 | 0.925 | 0.920 | 0.919 |
| CLIP 類似度                 | 0.304 | 0.302 | 0.291 | 0.273 | 0.271 | 0.262 | 0.258 |

構造整合は 2.0 で頭打ちになるが、プロンプト追従はそこを越えても下がり続ける。この掃引も 7 画像から 10 段階ずつなので、全 70 点を独立標本にはできない。画像ごとに系列内で順位相関を求めると、構造整合は中央値  $\rho = +0.915$  で 7 画像すべてが正、プロンプト追従は中央値  $\rho = -0.673$  で 7 画像中 5 画像が負だった。向きが逆であることは表からも画像内の相関からも一貫しているが、プロンプト追従の低下は統計的には言い切れない (Wilcoxon の符号順位検定で  $p = 0.078$ )。標本が 7 画像しかないためである。

両方を正規化して和を最大化する scale を入力ごとに求めると 0.6 から 3.0 まで散る。構造整合だけで選ぶと全入力に 1.6~3.0 に集まるのに、両立を求めると入力ごとに異なる。ただし第 4.3 節の予想は「最適な scale は入力の密度によって動く」であり、密度との対応は有意には出ない (両立させた最適値で  $\rho = -0.34$ 、構造整合だけの最適値でも  $\rho = -0.63 \cdot p = 0.13$ )。予想は外れたままで、成立したのは「入力によって動く」という弱めた形までである。

条件の強さは、増やすほど良い資源ではない。第 4.2 節と第 5 節で見たのは資源が足りない側の破綻、本節で見たのは資源が過剰な側の代償である。密度が低い入力でも scale を上げれば構造整合は改善するが (synth\_sparse は 0.333→0.877)、条件地図が空になった入力は掃引していない。残差に条件由来の成分が無い以上 (第 5 節)、増幅しても指定は戻らないはずである。密度と scale は、条件の実効的な強さという一つの量に対する別々の支払い方であり、最適値は入力の中ではなく、入力の構造と何を達成したいかの組み合わせの中にある。

筆者は大学院で、資源を増やすほど成績が上がるとは限らず、その価値がタスクの構造との整合で決まることを扱っている。本レポートの条件の強さも、その意味での資源の一例である。有用なのは逆向きの示唆で、手法の既定値は目的が一つに定まっているときの値であり、測る指標を変えれば妥当性も変わる。controlnet\_conditioning\_scale = 1.0 は構造整合では明らかに弱い、両指標を正規化した和の平均では 10 段階中で最も高い。既定値が「弱すぎる」と読めたのは、片方しか測っていなかったからである。

## 9. 生成AI利用

使用した生成AIは Claude (Anthropic) で、対話と自律実行の両方の形で用いた。何を実験し、何を主張し、何を撤回するかは筆者が決め、生成AIにはその方針に沿った作業を行わせた。主な用途は末尾の提出物3に挙げる。

生成AIが書いたものをそのまま採らないという方針を先に決め、下書きは節ごとに主張が測定に対応しているかを確認、対応しない箇所は撤回するか書き直した。あわせて機械的な検査を用意した。stats\_report.py は本文が挙げる統計量を節ごとに再現し、audit\_numbers.py は本文中の数値がその出力か results/\*.csv に現れるかを照合して、現れないものを「出所不明」として挙げる。ほかに本文の論理のつながり (audit\_flow.py) と、本文・README・コードの記述の食い違い (audit\_consistency.py) も機械で見ている。

自分で修正・判断した箇所を挙げる。いずれも生成AIの出力を疑って一次資料に当たった結果である。論文の扱いでは「解像度依存の重みが diffusers の既定では効いていない」と食い違いを指摘していたが、読み直すとその重みは classifier-free guidance の特定の場合の手当てで、guess\_mode が正しく対応していた。実装の不備ではないので撤回した。書誌も同様に照合しており、会議名は「CVPR 2023」と提示されたものを Crossref で DOI から ICCV 2023 に直している。

主張の根拠に関わる訂正が二件ある。「条件地図が空のとき edge F1 が例外なく 0 になる」と「空の条件地図の残差が参照値とほぼ一致する」を、いずれも破綻の証拠として書いていた。どちらも指標や比較の定義から出る値で、測定ではない (理由は第 4.2 節と第 5 節)。撤回し、偶然一致の水準との比較と、密度が正の領域での減衰に置き換えた。第 5 節の系列内相関も、密度 0 の点を含んだまま「全画像が正」としていたのを、密度が実際に動く画像に限った値へ直している。

検定の設計も直した。標本数の水増しとコントラストの層別、改善前後で参照地図が違う点である (いずれも第 6 節)。生成AIが提示した測定値は根拠にせず、同じ測定を実装し直して再現を確認してから載せた。再現できなかった数値は外した。

## 参考文献

- [1] Lvmin Zhang, Anyi Rao, Maneesh Agrawala. "Adding Conditional Control to Text-to-Image Diffusion Models." Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 3813–3824. DOI: 10.1109/ICCV51070.2023.00355 (Crossref・DBLP・Semantic Scholar で照合。supplementary は CVF 公開版)
- [2] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Björn Ommer. "High-Resolution Image Synthesis with Latent Diffusion Models." CVPR, 2022, pp. 10674–10685. DOI: 10.1109/CVPR52688.2022.01042 (Crossref で照合)
- [3] Hugging Face. "Metal Performance Shaders (MPS)." diffusers documentation.  
<https://huggingface.co/docs/diffusers/en/optimization/mps>
- [4] diffusers v0.31.0 実装 (models/controlnet.py、models/attention\_processor.py)
- [5] Stéfan van der Walt et al. "scikit-image: image processing in Python." PeerJ 2:e453, 2014. DOI: 10.7717/peerj.453 (画像ごとの帰属はリポジトリの LICENSE\_DATA.md)
- [6] BLAS Level 3 GEMM の仕様 (netlib LAPACK ドキュメント)。BETA が 0 のとき加算元 C は入力として設定されていなくてよい: "When BETA is supplied as zero then C need not be set on input."  
<https://www.netlib.org/lapack/explore-html/>

## 提出物

1. 本レポート (本 PDF)。
2. ソースコード : <https://github.com/komeiall14/controlnet-failure-analysis>
3. 生成AI利用ログ (詳細は第 9 節)。使用した生成AI は Claude (Anthropic)。主な用途は実験コードの実装、統計処理と作図、本レポートの下書き、完成前の批判的レビュー。自分で修正・判断した箇所は、何を実験し何を主張し何を撤回するか決定と、第 9 節の差し戻しである。